

# Workflow and architecture patterns for working together 1: Workflows

Jacob Archambault

May 15, 2026

# Outline

## 1 Current challenges and their canonical solutions

# Outline

- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
  - Communication anti-pattern: local optimization
  - Throughput
  - Throughput anti-pattern: Gandalf vs. the Balrog

# Outline

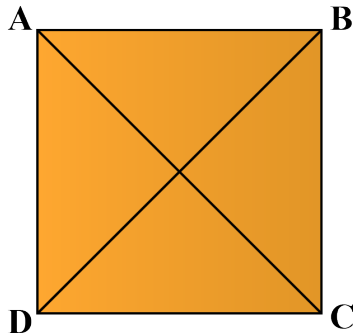
- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
  - Communication anti-pattern: local optimization
  - Throughput
  - Throughput anti-pattern: Gandalf vs. the Balrog
- 3 Branching and continuous integration
  - Definition
  - CI Branching patterns
  - Trade-offs
  - Enablers
  - Benefits

# Outline

- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
  - Communication anti-pattern: local optimization
  - Throughput
  - Throughput anti-pattern: Gandalf vs. the Balrog
- 3 Branching and continuous integration
  - Definition
  - CI Branching patterns
  - Trade-offs
  - Enablers
  - Benefits
- 4 Conclusion

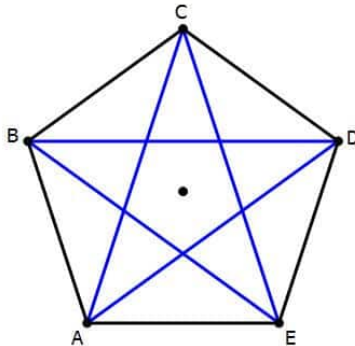
## 1975: The Mythical Man Month, Fred Brooks

- number of direct communication paths for  $n$  individuals= $n(n-1)/2$



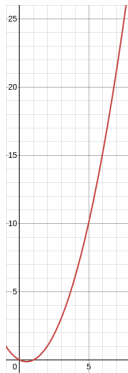
## 1975: The Mythical Man Month, Fred Brooks

- number of direct communication paths for  $n$  individuals= $\frac{n(n-1)}{2}$



## 1975: The Mythical Man Month, Fred Brooks

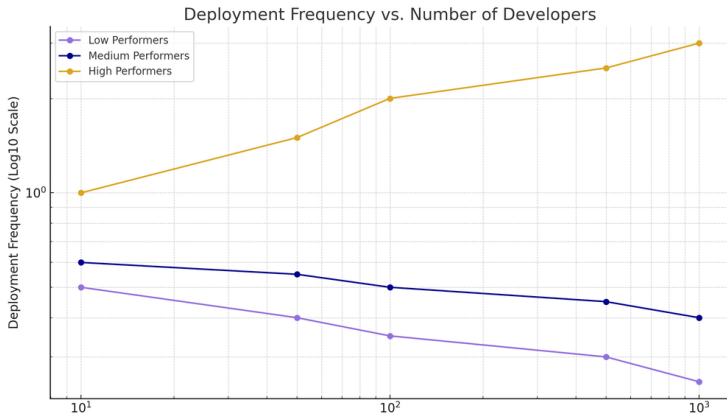
- number of direct communication paths for  $n$  individuals= $n(n-1)/2$





## The Mythical Man Month, Fred Brooks (cont.)

- corollary: adding more people to a project can lead not only to diminishing returns on delivery speed, but to objectively less work being completed



# Challenges

- coordinating multiple teams working in the same repository

# Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing

# Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code

# Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays

# Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions

# Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks

# Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer



# Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs

# Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs
  - single point of failure risk

# Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs
  - single point of failure risk
  - risk of cross-project interference

# Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs
  - single point of failure risk
  - risk of cross-project interference

# Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver

# Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams

# Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow

# Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow
- database coupling → Bezos API mandate/“Microservices”



# Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow
- database coupling → Bezos API mandate/“Microservices”

# Outline

- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
    - Communication anti-pattern: local optimization
    - Throughput
    - Throughput anti-pattern: Gandalf vs. the Balrog
- 3 Branching and continuous integration
  - Definition
  - CI Branching patterns
  - Trade-offs
  - Enablers
  - Benefits
- 4 Conclusion

## 1967: Conway's law

*'Organizations which design systems [...] are constrained to produce designs which are copies of the communication structures of these organizations.'* - Melvin Conway, 'How do Committees Invent?' *Datamation*, 1967

## Conway's law: examples

- 'If you have four groups working on a compiler, you'll get a 4-pass compiler' - The New Hacker's Dictionary, 1996

## Conway's law: examples

- 'If you have four groups working on a compiler, you'll get a 4-pass compiler' - The New Hacker's Dictionary, 1996
- front-end [devs] business layer [backend devs] monolithic database [DBA team]

## Conway's law: examples

- 'If you have four groups working on a compiler, you'll get a 4-pass compiler' - The New Hacker's Dictionary, 1996
- front-end [devs] business layer [backend devs] monolithic database [DBA team]
- a web api controller [manager] delegates most business logic to business classes [developers] which are part of the same in-memory process [team], while serving as a single entry-point for wider cross-network [cross-team] communication.

## Conway's law: more examples

- separate .csproj projects for tests written by QA

## Conway's law: more examples

- separate .csproj projects for tests written by QA
- separate development and config repositories for the same transaction signifying low communication between development and operations



## Conway's law: more examples

- separate .csproj projects for tests written by QA
- separate development and config repositories for the same transaction signifying low communication between development and operations
- all business documentation in the same repository, separate from the repositories that the documentation is for

## Conway's law: more examples

- separate .csproj projects for tests written by QA
- separate development and config repositories for the same transaction signifying low communication between development and operations
- all business documentation in the same repository, separate from the repositories that the documentation is for

# Conway's law: corollaries

- Conway's law can't be fought

## Conway's law: corollaries

- Conway's law can't be fought
- Teams working in the same part of the same codebase at the same time are the same team

## Conway's law: corollaries

- Conway's law can't be fought
- Teams working in the same part of the same codebase at the same time are the same team
  - separate the codebases for improved velocity

## Conway's law: corollaries

- Conway's law can't be fought
- Teams working in the same part of the same codebase at the same time are the same team
  - separate the codebases for improved velocity
  - join the teams for improved code sharing and communication (Inverse conway maneuver)

## Conway's law: corollaries

- Conway's law can't be fought
- Teams working in the same part of the same codebase at the same time are the same team
  - separate the codebases for improved velocity
  - join the teams for improved code sharing and communication (Inverse conway maneuver)
  - branching-based strategies for team management compromise between the above approaches without their benefits

# Outline

- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
  - **Communication anti-pattern: local optimization**
  - Throughput
  - Throughput anti-pattern: Gandalf vs. the Balrog
- 3 Branching and continuous integration
  - Definition
  - CI Branching patterns
  - Trade-offs
  - Enablers
  - Benefits
- 4 Conclusion



## communication anti-pattern: local optimization

- example: separate (DevOps, QA, operations, support) team

## communication anti-pattern: local optimization

- example: separate (DevOps, QA, operations, support) team
- leads to bottlenecks

## communication anti-pattern: local optimization

- example: separate (DevOps, QA, operations, support) team
- leads to bottlenecks
- delays communication

## communication anti-pattern: local optimization

- example: separate (DevOps, QA, operations, support) team
- leads to bottlenecks
- delays communication
- discourages cross-functional collaboration

## communication anti-pattern: local optimization

- example: separate (DevOps, QA, operations, support) team
- leads to bottlenecks
- delays communication
- discourages cross-functional collaboration
- punishes untracked work (e.g. thorough code reviews/dev testing, helping blocked colleagues)

# Solution

- autonomous, cross-functional teams with 't-shaped' employees

# Solution

- autonomous, cross-functional teams with 't-shaped' employees
- communities of practice

# Solution

- autonomous, cross-functional teams with 't-shaped' employees
- communities of practice



## Revisiting Conway's law: defining a team

- A *team* is the minimal socio-technical unit that can deliver a production change with no additional coordination beyond mandated governance or custodial controls.

## Revisiting Conway's law: defining a team

- A *team* is the minimal socio-technical unit that can deliver a production change with no additional coordination beyond mandated governance or custodial controls.
  - pull requests

## Revisiting Conway's law: defining a team

- A *team* is the minimal socio-technical unit that can deliver a production change with no additional coordination beyond mandated governance or custodial controls.
  - pull requests
  - testing

## Revisiting Conway's law: defining a team

- A *team* is the minimal socio-technical unit that can deliver a production change with no additional coordination beyond mandated governance or custodial controls.
  - pull requests
  - testing
  - deployment

## Revisiting Conway's law: defining a team

- A *team* is the minimal socio-technical unit that can deliver a production change with no additional coordination beyond mandated governance or custodial controls.
  - pull requests
  - testing
  - deployment

# Outline

- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
  - Communication anti-pattern: local optimization
  - **Throughput**
  - Throughput anti-pattern: Gandalf vs. the Balrog
- 3 Branching and continuous integration
  - Definition
  - CI Branching patterns
  - Trade-offs
  - Enablers
  - Benefits
- 4 Conclusion

# Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput

# Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput
- Decrease:



# Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput
- Decrease:
  - operating costs

# Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput
- Decrease:
  - operating costs
  - inventory

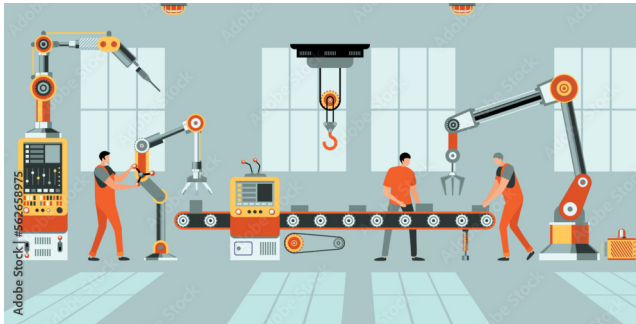
# Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput
- Decrease:
  - operating costs
  - inventory
  - scrap

# Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput
- Decrease:
  - operating costs
  - inventory
  - scrap
- Remove bottlenecks

## Goldratt's theory of constraints (cont.)



# 2013: The State of DevOps Report

- a series of surveys of over 36,000 professionals

## 2013: The State of DevOps Report

- a series of surveys of over 36,000 professionals
- Led by a PhD statistician, Nicole Forsgren, from 2013-2019

## 2013: The State of DevOps Report

- a series of surveys of over 36,000 professionals
- Led by a PhD statistician, Nicole Forsgren, from 2013-2019
- Uses *cluster analysis* to discover groupings - has no prior understanding of what counts as good or bad.



# The State of DevOps Report: throughput metrics

- lead time for changes

# The State of DevOps Report: throughput metrics

- lead time for changes
- deployment frequency

# The State of DevOps Report: stability metrics

- change failure rate

# The State of DevOps Report: stability metrics

- change failure rate
- mean time to restore

## The State of DevOps Report: results

*'Astonishingly, these results demonstrate that there is no trade-off between improving performance and achieving higher levels of stability and quality. Rather, high performers do better at all of these measures. This is precisely what the Agile and Lean movements predict, but much dogma in our industry still rests on the false assumption that moving faster means trading off against other performance goals, rather than enabling and reinforcing them.'*  
- Nicole Forsgren, Jez Humble, and Gene Kim, Accelerate (2017), p. 14

# The State of DevOps Report: results

When compared to low performers, elite performers realize

**127x**

faster lead  
time

**182x**

more  
deployments  
per year

**8x**

lower change  
failure rate

**2293x**

faster failed  
deployment  
recovery times

- 2024 State of DevOps Report

# Outline

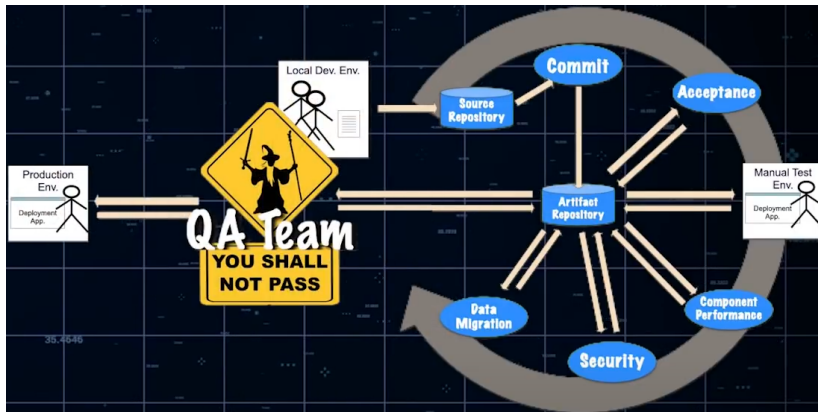
- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
  - Communication anti-pattern: local optimization
  - Throughput
  - Throughput anti-pattern: Gandalf vs. the Balrog
- 3 Branching and continuous integration
  - Definition
  - CI Branching patterns
  - Trade-offs
  - Enablers
  - Benefits
- 4 Conclusion

## Throughput anti-pattern: Gandalf vs. the Balrog





## Throughput anti-pattern: Gandalf vs. the Balrog



# Throughput anti-pattern: Gandalf vs. the Balrog

- change approval boards

# Throughput anti-pattern: Gandalf vs. the Balrog

- change approval boards
- pre-merge code reviews

# Throughput anti-pattern: Gandalf vs. the Balrog

- change approval boards
- pre-merge code reviews
- 'bridge-to-nowhere' pipelines

# Throughput anti-pattern: Gandalf vs. the Balrog

- change approval boards
- pre-merge code reviews
- 'bridge-to-nowhere' pipelines
- these all increase lead time

# Throughput anti-pattern solutions: aggressively parallelize work

- pair programming

# Throughput anti-pattern solutions: aggressively parallelize work

- pair programming
- parallelized pipeline builds for 'wide' pipelines

# Throughput anti-pattern solutions: aggressively parallelize work

- pair programming
- parallelized pipeline builds for 'wide' pipelines
- one build artifact for all pipeline stages



# Throughput anti-pattern solutions: aggressively parallelize work

- pair programming
- parallelized pipeline builds for 'wide' pipelines
- one build artifact for all pipeline stages
- share responsibility

# Defining throughput for our project

- Avoid 'thrashing' metrics

## Defining throughput for our project

- Avoid 'thrashing' metrics
- *Throughput* - the total lapsed time from local commit to production deployment

## Defining throughput for our project

- Avoid 'thrashing' metrics
- *Throughput* - the total lapsed time from local commit to production deployment
- *Throughput\** - the total lapsed time from local commit to user acceptance testing handoff

## Defining throughput for our project

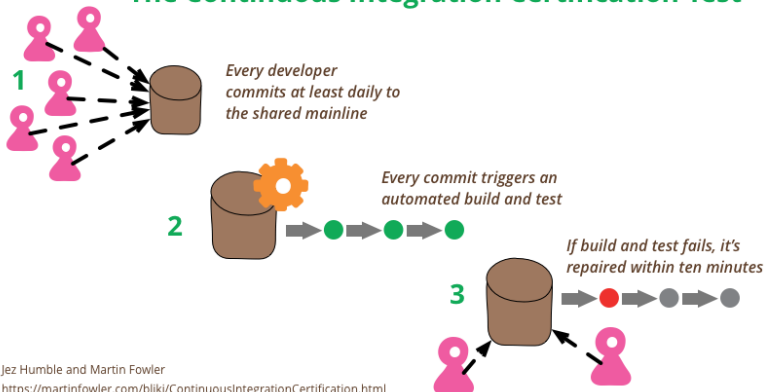
- Avoid 'thrashing' metrics
- *Throughput* - the total lapsed time from local commit to production deployment
- *Throughput\** - the total lapsed time from local commit to user acceptance testing handoff

# Outline

- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
  - Communication anti-pattern: local optimization
  - Throughput
  - Throughput anti-pattern: Gandalf vs. the Balrog
- 3 Branching and continuous integration
  - Definition
  - CI Branching patterns
  - Trade-offs
  - Enablers
  - Benefits
- 4 Conclusion

# Defining continuous integration

## The Continuous Integration Certification Test



Jez Humble and Martin Fowler

<https://martinfowler.com/bliki/ContinuousIntegrationCertification.html>

# Outline

- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
  - Communication anti-pattern: local optimization
  - Throughput
  - Throughput anti-pattern: Gandalf vs. the Balrog
- 3 Branching and continuous integration
  - Definition
  - CI Branching patterns
  - Trade-offs
  - Enablers
  - Benefits
- 4 Conclusion



# Continuous integration branching patterns

# Continuous integration branching patterns

- don't

# Continuous integration branching patterns

- don't
- as little as possible, for as short a time as possible, at least daily

# Continuous integration branching patterns

- don't
- as little as possible, for as short a time as possible, at least daily

# Outline

- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
  - Communication anti-pattern: local optimization
  - Throughput
  - Throughput anti-pattern: Gandalf vs. the Balrog
- 3 Branching and continuous integration
  - Definition
  - CI Branching patterns
  - **Trade-offs**
  - Enablers
  - Benefits
- 4 Conclusion

## Continuous integration: trade-offs

*Continuous Integration applies a different trigger for integration - you integrate whenever you've made a hunk of progress on the feature and your branch is still healthy. There's no expectation that the feature be complete, just that there's been a worthwhile amount of changes to the codebase. The rule of thumb is that "everyone commits to the mainline every day", or more precisely: you should never have more than a day's work sitting unintegrated in your local repository.*

*- Martin Fowler, "Branching Patterns"*

### **Feedback comes late**

Too late to change anything substantial

### **Low quality feedback**

Through comments, lacking nuance

### **"My code"**

Person owns area of code; key-person dependencies

### **Individual coding styles**

Harder to have standards

### **Infrequent integration**

Only integrate at the end of story/feature

### **Easy to ignore a failing build**

Because it runs on an isolated branch

### **Dread large refactorings**

Because they are likely to cause merge conflicts

### **People work in isolation**

Harder to spot if someone needs help

### **Feedback comes in real time**

As you're writing code or even before

### **Better quality feedback**

Through face-to-face discussion

### **"Our code"**

Collective code ownership

### **Team coding style**

Easier to establish standards when pairing

### **Continuous Integration**

Integrate as soon as code is pushed

### **We get used to not breaking things**

Broken build on master is the team top priority

### **Easier to tackle large refactorings**

As the rest of the team is always up to date

### **Visibility of what everyone is doing**

Easier to spot if someone needs help

# Outline

- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
  - Communication anti-pattern: local optimization
  - Throughput
  - Throughput anti-pattern: Gandalf vs. the Balrog
- 3 Branching and continuous integration**
  - Definition
  - CI Branching patterns
  - Trade-offs
  - Enablers**
  - Benefits
- 4 Conclusion



# Continuous integration: Enablers

- fast PR turnaround

# Continuous integration: Enablers

- fast PR turnaround
- improving CI pipeline runtime

# Continuous integration: Enablers

- fast PR turnaround
- improving CI pipeline runtime
- team discipline

# Continuous integration: Enablers

- fast PR turnaround
- improving CI pipeline runtime
- team discipline

# Outline

- 1 Current challenges and their canonical solutions
- 2 DevOps 201
  - Communication
  - Communication anti-pattern: local optimization
  - Throughput
  - Throughput anti-pattern: Gandalf vs. the Balrog
- 3 Branching and continuous integration
  - Definition
  - CI Branching patterns
  - Trade-offs
  - Enablers
  - **Benefits**
- 4 Conclusion

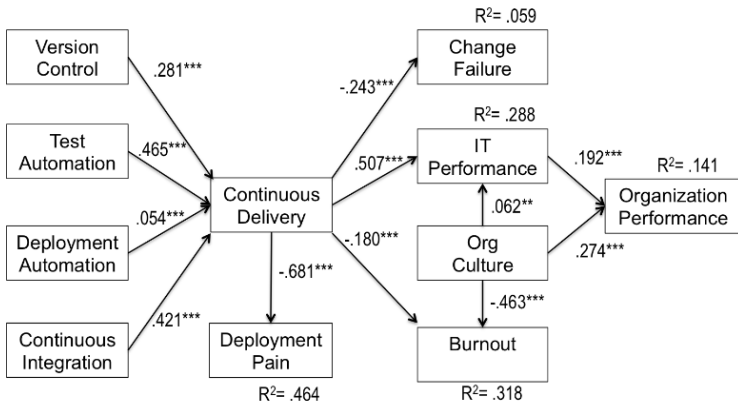


Figure 2. Emergent Model

- Forsgren and Humble (2016), p. 10

## Continuous integration: benefits (cont.)

- High performers have the shortest integration times and branch lifetimes, with branch life and integration typically lasting hours.

## Continuous integration: benefits (cont.)

- High performers have the shortest integration times and branch lifetimes, with branch life and integration typically lasting hours.
- Low performers have the longest integration times and branch



## Continuous integration: benefits (cont.)

- High performers have the shortest integration times and branch lifetimes, with branch life and integration typically lasting hours.
- Low performers have the longest integration times and branch lifetimes, with branch life and integration typically lasting days.

## Continuous integration: benefits (cont.)

- High performers have the shortest integration times and branch lifetimes, with branch life and integration typically lasting hours.
- Low performers have the longest integration times and branch lifetimes, with branch life and integration typically lasting days.
- These differences are statistically significant
  - 2017 State of DevOps Report, p. 41

# Conclusion

- Reduce goal misalignment

# Conclusion

- Reduce goal misalignment
- stop people from waiting on each other

# Conclusion

- Reduce goal misalignment
- stop people from waiting on each other
- work in small batches

# Conclusion

- Reduce goal misalignment
- stop people from waiting on each other
- work in small batches
- That's mostly it

# Conclusion

*'DevOps is whatever you do to bridge friction created by silos, and all the rest is engineering' - Patrick Debois, Puppet State of DevOps report 2021*

# Workflow and architecture patterns for working together 1: Workflows

Jacob Archambault

May 15, 2026